# Dashboard Speed Optimization

| Date | August 5, 2026 |
| --- | --- |

## Suricata Dashboard: End-to-End Performance Optimization Notes

This document captures the complete chronological record of architectural changes, diagnostic steps, and performance improvements implemented to optimize the Suricata Monitoring Dashboard.

### ***Phase 1: Performance Instrumentation & Bottleneck Detection***

To diagnose UI lag and isolate slow execution blocks without relying on slower UI debugging tools, precise timing instrumentation was introduced using Python's `time.time()` and standard console printing (`print()`).

• Console-Based Timing Logs: Added granular timestamps around data ingestion, rule catalog loading, and database fetches in the main render loop.

• Row-Click Diagnostic Tracking: Inserted execution timers around row selections and detail panel lookups to measure exact latency when interacting with the live alert table.

• Terminal Visibility: Configured logs to stream directly into the active `streamlit run` terminal window, allowing instant verification without restarting the application server.

### ***Phase 2: UI Isolation via Streamlit Fragments (`@st.fragment`)***

The primary source of lag in the original dashboard was cascading re-renders—clicking an alert row or changing a table page forced the *entire* application (including heavy analytics charts and database re-queries) to redraw.

• Fragment Decoupling: Wrapped the alerts table and detail pane inside an isolated `@st.fragment` execution context (`render_alerts_table_and_details`).

• Analytics Separation: Moved KPI rows, trends, and charts into separate fragment scopes (`render_alerts_analytics`).

• Targeted Reruns: Configured table selections and pagination to execute independent local reruns, restricting DOM updates strictly to the active table/detail view and dropping table-render latency to as low as 0.03s–0.10s.

### ***Phase 3: Rule Catalog Caching & O(1) Lookups***

Enriching Suricata alerts with rule descriptions and metadata (`msg`, `classtype`, `rev`) was initially creating overhead when cross-referencing thousands of events.

- In-Memory SID Catalog Caching: Implemented `@st.cache_data` on rule-loading modules to parse JSON signature files (`suricata_community_sample.json`, `suricata_et_rules.json`) exactly once.
- O(1) Dictionary Mapping: Converted signature lookups into direct dictionary indexing (`sid_dict[sid]`), successfully caching over 88,391 SIDs in memory on startup.
- Bypassing Redundant Disk I/O: Eliminated repeated file reads on subsequent UI refreshes and row clicks, ensuring instant lookup speeds for alert metadata.

***Phase 4: Database Ingestion & Fetch Optimizations***

Managing SQLite operations and dataframe transformations efficiently prevented memory bloat and connection bottlenecks during high-frequency log updates.

- WAL Mode Activation: Configured SQLite with Write-Ahead Logging (`PRAGMA journal_mode=WAL;`) to handle concurrent reads and writes smoothly during background log ingestion.
- Adjustable Fetch Limits: Added sidebar sliders to let users control maximum database fetch limits (`250` to `2000` rows), preventing massive dataframe processing overhead on low-spec client instances.
- Clean Dependency Resolution: Standardized module imports between `app.py` and `modules/data_loader.py`, aligning helper functions (`load_rule_catalog`, `load_sid_catalog`, and `prepare_alerts_dataframe`) to work seamlessly together without import errors.

***Final Performance Outcomes***

With these optimizations fully integrated, the dashboard achieves:

- Cold Startup & Catalog Build: ~88,300+ SIDs cached instantly on launch.
- Fragment Execution: Table selections and expansions render in 0.03s to 0.10s.
- Full Cycle Loops: Total dashboard render and ingest cycles consistently complete under 1.5s to 2.0s.